

ANCILLARY REVENUE ANALYSIS - SOLSTICE OPAL HOTEL



Introduction

The purpose of this analysis is to identify which guest segments generate the most ancillary revenue (spa, dining, and activities) at Solstice Opal Hotel, and to provide actionable recommendations that leadership can use to maximise this revenue stream. The analysis draws on three datasets: guest profiles, stay details, and ancillary spend records.

```
#import libraries
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import re
import warnings
warnings.filterwarnings('ignore')
```

Download and Explore the Data

```
#Load the data
stay_details = pd.read_csv('da_sample_stay_details.csv')
guest_profiles = pd.read_csv('da_sample_guest_profiles.csv')
spend = pd.read_csv('da_sample_ancillary_spend.csv')
```

```
# Rows and columns for each dataset
print("Data loaded")
print(f"    guest_profiles : {guest_profiles.shape[0]} rows,
{guest_profiles.shape[1]} columns")
print(f"    stay_details   : {stay_details.shape[0]} rows, {stay_details.shape[1]}
columns")
print(f"    ancillary_spend: {spend.shape[0]} rows, {spend.shape[1]} columns")
```

Data loaded

```
guest_profiles : 400 rows, 3 columns
stay_details   : 600 rows, 6 columns
ancillary_spend: 800 rows, 3 columns
```

```
#Guest_profile Check
```

```
print("\n" + "="*60)
print("GUEST PROFILES")
print("="*60)
print(guest_profiles.head())
print("\nData types:")
print(guest_profiles.dtypes)
print("\nMissing values per column:")
print(guest_profiles.isnull().sum())
```

```
=====
GUEST PROFILES
=====
```

	guest_id	loyalty_tier	marketing_consent
0	1	NaN	True
1	2	Platinum	False
2	3	Silver	True
3	4	Silver	False
4	5	NaN	True

Data types:

```
guest_id          int64
loyalty_tier      object
marketing_consent  bool
dtype: object
```

Missing values per column:

```
guest_id          0
loyalty_tier      193
marketing_consent  0
dtype: int64
```

#Stay Details Check

```
print('\n' + '=' * 60)
print('STAY DETAILS')
print('=' * 60)
print(stay_details.head())
print('\nData Types:')
print(stay_details.dtypes)
print('\nMissing Values per column:')
print(stay_details.isnull().sum())
```

```
=====
STAY DETAILS
=====
```

	guest_id	check_in_date	...	reason_for_stay	number_of_guests
0	238	2025-10-25 12:09:35.704163	...	business	4
1	129	2025-11-07 12:09:35.704163	...	business	2
2	332	2025-09-30 12:09:35.704163	...	business	1
3	153	2025-09-06 12:09:35.704163	...	business	4
4	305	2025-11-23 12:09:35.704163	...	business	2

[5 rows x 6 columns]

Data Types:

```
guest_id          int64
check_in_date     object
stay_id           object
booking_channel   object
reason_for_stay   object
number_of_guests  int64
dtype: object
```

Missing Values per column:

```
guest_id          0
check_in_date     0
stay_id           0
booking_channel   0
reason_for_stay   0
number_of_guests  0
```

#Ancillary Spend

```

print('\n' + '=' * 60)
print('ANCILLARY SPEND')
print('=' * 60)
print(spend.head())
print('\Data Types:')
print(spend.dtypes)
print('\nMissing Value per Column:')
print(spend.isnull().sum())

```

```

=====
ANCILLARY SPEND
=====
  guest_id category  amount_spent
0      170D   Dining           NaN
1      124D   Dining         49.72
2      357S     Spa         14.43
3       35D   Dining        330.85
4       87D   Dining           NaN
\Data Types:
guest_id      object
category      object
amount_spent  float64
dtype: object

Missing Value per Column:
guest_id      0
category      0
amount_spent   79
dtype: int64

```

DATA VALIDATION AND CLEANING

```

#---Guest Profile---
print("\n" + "="*60)
print("VALIDATION: GUEST PROFILES")
print("="*60)
print(f"\nguest_id duplicates : {guest_profiles['guest_id'].duplicated().sum()}")
print(f"guest_id nulls      : {guest_profiles['guest_id'].isnull().sum()}")
print(f"guest_id range      : {guest_profiles['guest_id'].min()} to
{guest_profiles['guest_id'].max()}")
print(f"\nloyalty_tier unique
values:\n{guest_profiles['loyalty_tier'].value_counts(dropna=False)}")
print(f"\nmarketing_consent
values:\n{guest_profiles['marketing_consent'].value_counts(dropna=False)}")

```

```

=====
VALIDATION: GUEST PROFILES
=====

```

```

guest_id duplicates : 0
guest_id nulls      : 0
guest_id range      : 1 to 400

```

```

loyalty_tier unique values:
loyalty_tier

```

```

NaN          193
Silver       107
Gold         62
Platinum     38
Name: count, dtype: int64

```

```

marketing_consent values:

```

```

marketing_consent
True          315
False         85
Name: count, dtype: int64

```

```
#---Stay Details---

print("\n" + "="*60)
print("VALIDATION: STAY DETAILS")
print("="*60)

#guest_id in stay_details vs guest_id in guest_profile
guests_only_in_stays = set(stay_details['guest_id']) -
set(guest_profiles['guest_id'])
print(f"\nGuest IDs in stays but NOT in profiles: {len(guests_only_in_stays)}")

# stay_id duplicates
print(f"stay_id duplicates: {stay_details['stay_id'].duplicated().sum()}")

#booking_channel
print(f"\nbooking_channel
values:\n{stay_details['booking_channel'].value_counts(dropna=False)}")

#reason_for_stay
print(f"\nreason_for_stay
values:\n{stay_details['reason_for_stay'].value_counts(dropna=False)}")

# --- number_of_guests ---
# We check for zero or negative values – a stay with 0 guests makes no sense.
print(f"\nnumber_of_guests value
counts:\n{stay_details['number_of_guests'].value_counts().sort_index()}")
print(f"Zero or negative number_of_guests: {(stay_details['number_of_guests'] <=
0).sum()}")

# --- check for overlapping stays ---
# We convert check_in_date to a proper datetime format first.
stay_details['check_in_date'] = pd.to_datetime(stay_details['check_in_date'])
stays_sorted = stay_details.sort_values(['guest_id', 'check_in_date'])

#compare each stay's date to the PREVIOUS stay's date.
stays_sorted['prev_checkin'] = stays_sorted.groupby('guest_id')
['check_in_date'].shift(1)

# Calculate the gap in days between consecutive stays for the same guest
stays_sorted['days_gap'] = (stays_sorted['check_in_date'] -
stays_sorted['prev_checkin']).dt.days

# Flag exact same-day check-ins
overlapping = stays_sorted[stays_sorted['days_gap'] == 0]
print(f"\nOverlapping stays (same check-in date, same guest): {len(overlapping)}")

# Flag stays within 1 day of each other
```

```
close_stays = stays_sorted[(stays_sorted['days_gap'] >= 0) &
(stays_sorted['days_gap'] <= 1)]
print(f"Stays within 1 day of each other (same guest): {len(close_stays)}")
```

```
=====
VALIDATION: STAY DETAILS
=====
```

Guest IDs in stays but NOT in profiles: 0

stay_id duplicates: 29

booking_channel values:

booking_channel

website 267

travel_agent 219

corporate 114

Name: count, dtype: int64

reason_for_stay values:

reason_for_stay

leisure 305

business 295

Name: count, dtype: int64

number_of_guests value counts:

number_of_guests

1 296

2 220

3 58

4 26

Name: count, dtype: int64

Zero or negative number_of_guests: 0

```

print("\n" + "="*60)
print("VALIDATION: ANCILLARY SPEND")
print("="*60)

print(f"\nActual column names in spend: {list(spend.columns)}")

# --- guest_id values have letter suffixes ---
print(f"\nSample guest_id values: {spend['guest_id'].head(10).tolist()}")

# str[-1] extracts the last character of every string in the column
print(f"\nLast character (suffix) counts:")
print(spend['guest_id'].str[-1].value_counts())

# Extract just the numeric part using regex
spend['guest_id_numeric'] = spend['guest_id'].str.extract(r'(\d+)').astype(float)
print(f"\nAfter stripping suffix - ID range: {spend['guest_id_numeric'].min()} to {spend['guest_id_numeric'].max()}")

# Check if any of these IDs don't exist in the guest profiles
unmatched = set(spend['guest_id_numeric'].dropna().astype(int)) -
set(guest_profiles['guest_id'])
print(f"IDs in spend not found in profiles (after cleaning): {len(unmatched)}")

# --- category ---
print(f"\ncategory values:\n{spend['category'].value_counts(dropna=False)}")

# --- amount_spent ---
print(f"\namount_spent nulls      : {spend['amount_spent'].isnull().sum()}")
print(f"amount_spent negatives: {(spend['amount_spent'] < 0).sum()}")
print(f"amount_spent zeros      : {(spend['amount_spent'] == 0).sum()}")
print(f"\namount_spent statistics:\n{spend['amount_spent'].describe()}")

```

```

=====
VALIDATION: ANCILLARY SPEND
=====

```

Actual column names in spend: ['guest_id', 'category', 'amount_spent']

Sample guest_id values: ['170D', '124D', '357S', '35D', '87D', '134D', '196D', '274S', '201D', '369D']

Last character (suffix) counts:

guest_id	
D	424
S	256
A	120


```
Name: count, dtype: int64
```

```
After stripping suffix – ID range: 1.0 to 399.0
```

```
IDs in spend not found in profiles (after cleaning): 0
```

```
category values:
```

```
category
```

```
Dining      424
```

```
Spa         256
```

```
Activities   120
```

```
Name: count, dtype: int64
```

```
amount_spent nulls      : 79
```

```
amount_spent negatives: 0
```

DATA CLEANING

```

print("\n" + "="*60)
print("DATA CLEANING")
print("="*60)

# --- Clean 1: Fix ancillary_spend guest_id ---
spend['guest_id'] = spend['guest_id'].astype(str).str.extract(r'(\d+)').astype(int)
print(f"\n Stripped letter suffixes from ancillary_spend guest_id")
print(f"    Sample after fix: {spend['guest_id'].head(5).tolist()}")

# --- Clean 2: Drop rows with null amount_spent ---
rows_before = len(spend)
spend_clean = spend.dropna(subset=['amount_spent']).copy()
rows_after = len(spend_clean)
print(f"\n Dropped {rows_before - rows_after} rows with null amount_spent")
print(f"    Spend rows: {rows_before} → {rows_after}")

# --- Clean 3: Remove duplicate stay_ids ---
rows_before = len(stay_details)
stays_clean = stay_details.drop_duplicates(subset=['stay_id'], keep='first').copy()
rows_after = len(stays_clean)
print(f"\n Removed {rows_before - rows_after} duplicate stay_id rows")
print(f"    Stays rows: {rows_before} → {rows_after}")

# Make sure check_in_date is datetime in the clean version too
stays_clean['check_in_date'] = pd.to_datetime(stays_clean['check_in_date'])

# --- Clean 4: Fill null loyalty_tier with 'Non-member' ---
guests_clean = guest_profiles.copy()
nulls_before = guests_clean['loyalty_tier'].isnull().sum()
guests_clean['loyalty_tier'] = guests_clean['loyalty_tier'].fillna('Non-member')
print(f"\n Filled {nulls_before} null loyalty_tier values with 'Non-member'")
print(f"    Loyalty tier distribution after fix:")
print(f"    {guests_clean['loyalty_tier'].value_counts().to_dict()}")

```

```

=====
DATA CLEANING
=====

```

Stripped letter suffixes from ancillary_spend guest_id
 Sample after fix: [170, 124, 357, 35, 87]

Dropped 79 rows with null amount_spent
 Spend rows: 800 → 721

Removed 29 duplicate stay_id rows
 Stays rows: 600 → 571

```
Filled 193 null loyalty_tier values with 'Non-member'
```

```
Loyalty tier distribution after fix:
```

```
{'Non-member': 193, 'Silver': 107, 'Gold': 62, 'Platinum': 38}
```

MERGE THE THREE TABLES

```

print("\n" + "="*60)
print("MERGING TABLES")
print("="*60)

# --- Merge 1: Stays + Guest Profiles ---
merged = stays_clean.merge(guests_clean, on='guest_id', how='left')
print(f"\nMerged stays + guests: {merged.shape[0]} rows")

# --- Aggregate spend per guest ---
spend_agg = spend_clean.groupby('guest_id').agg(
    total_ancillary_spend = ('amount_spent', 'sum'),    # sum all amounts
    num_transactions      = ('amount_spent', 'count')  # count how many transactions
).reset_index()

print(f" Aggregated spend per guest: {spend_agg.shape[0]} guests with spend
records")
print(f"   Sample:\n{spend_agg.head()}")

# --- Merge 2: Add spend to the main table ---
full = merged.merge(spend_agg, on='guest_id', how='left')

# Guests with NO spend records will have NaN - replace with 0
# These are the "zero-spend" guests - they stayed but bought nothing.
full['total_ancillary_spend'] = full['total_ancillary_spend'].fillna(0)
full['num_transactions']      = full['num_transactions'].fillna(0)

print(f" Final merged table: {full.shape[0]} rows x {full.shape[1]} columns")
print(f"\nFinal table columns: {list(full.columns)}")
print(f"\nSample of final table:")
print(full[['guest_id', 'loyalty_tier', 'reason_for_stay', 'booking_channel', 'total_anc
illary_spend']].head(8))

```

```

=====
MERGING TABLES
=====

```

```

Merged stays + guests: 571 rows
Aggregated spend per guest: 324 guests with spend records
Sample:
  guest_id  total_ancillary_spend  num_transactions
0         1                   69.45                1
1         3                   157.81               3
2         4                   413.70               2
3         5                   293.00               3
4         6                    39.37               1

Final merged table: 571 rows x 10 columns

```

Final table columns: ['guest_id', 'check_in_date', 'stay_id', 'booking_channel', 'reason_for_stay', 'number_of_guests', 'loyalty_tier', 'marketing_consent', 'total_ancillary_spend', 'num_transactions']

Sample of final table:

	guest_id	loyalty_tier	reason_for_stay	booking_channel	total_ancillary_spend
0	238	Non-member	business	website	0.00
1	129	Non-member	business	corporate	82.84
2	332	Gold	business	website	178.22
3	153	Non-member	business	corporate	20.16
4	305	Silver	business	website	238.01
5	157	Silver	business	website	280.64
6	347	Silver	leisure	website	0.00

EXPLORATORY DATA ANALYSIS

```

print("\n" + "="*60)
print("EXPLORATORY ANALYSIS – KEY NUMBERS")
print("="*60)

# --- Headline numbers ---
total_revenue = full['total_ancillary_spend'].sum()
total_stays = len(full)
avg_per_stay = total_revenue / total_stays
zero_spend = (full['total_ancillary_spend'] == 0).sum()
zero_spend_pct = 100 * zero_spend / total_stays

print(f"\nHEADLINE NUMBERS")
print(f"    Total ancillary revenue : ${total_revenue:,.2f}")
print(f"    Total stays                : {total_stays}")
print(f"    Avg revenue per stay       : ${avg_per_stay:.2f}  -- THIS IS OUR KEY METRIC (AARS)")
print(f"    Zero-spend stays          : {zero_spend} ({zero_spend_pct:.1f}%)")

# --- By loyalty tier ---
tier_order = ['Non-member', 'Silver', 'Gold', 'Platinum']

tier_analysis = full.groupby('loyalty_tier')['total_ancillary_spend'].agg(
    avg_spend = 'mean',
    total_spend= 'sum',
    num_stays = 'count'
).reindex(tier_order) # .reindex() forces the rows to appear in our chosen order

print(f"\nBY LOYALTY TIER")
print(tier_analysis.round(2))

# --- By reason for stay ---
reason_analysis = full.groupby('reason_for_stay')['total_ancillary_spend'].agg(
    avg_spend = 'mean',
    total_spend= 'sum',
    num_stays = 'count'
)
print(f"\nBY REASON FOR STAY")
print(reason_analysis.round(2))

# --- By booking channel ---
channel_analysis = full.groupby('booking_channel')['total_ancillary_spend'].agg(
    avg_spend = 'mean',
    total_spend= 'sum',
    num_stays = 'count'
).sort_values('avg_spend', ascending=False)
print(f"\nBY BOOKING CHANNEL")
print(channel_analysis.round(2))

```

```
# --- By category (from spend_clean, not full) ---
# We use spend_clean here because we want per-transaction category data
cat_spend = spend_clean.groupby('category')
['amount_spent'].sum().sort_values(ascending=False)
print(f"\nTOTAL REVENUE BY CATEGORY")
print(cat_spend.round(2))

# --- Loyalty tier x reason for stay (multivariate) ---
pivot = full.groupby(['loyalty_tier', 'reason_for_stay'])
['total_ancillary_spend'].mean().unstack()
pivot = pivot.reindex(tier_order)
print(f"\n AVG SPEND BY TIER AND REASON FOR STAY")
print(pivot.round(2))
```

```
=====
EXPLORATORY ANALYSIS – KEY NUMBERS
=====
```

HEADLINE NUMBERS

```
Total ancillary revenue : $101,898.13
Total stays              : 571
Avg revenue per stay    : $178.46 -- THIS IS OUR KEY METRIC (AARS)
Zero-spend stays        : 68 (11.9%)
```

BY LOYALTY TIER

	avg_spend	total_spend	num_stays
loyalty_tier			
Non-member	141.39	37609.84	266
Silver	176.56	28779.84	163
Gold	191.24	17211.94	90
Platinum	351.86	18296.51	52

BY REASON FOR STAY

	avg_spend	total_spend	num_stays
reason_for_stay			
business	169.42	47438.09	280
leisure	187.15	54460.04	291

BY BOOKING CHANNEL

	avg_spend	total_spend	num_stays
booking_channel			
website	191.64	48100.66	251